

Extraction Workbench - Project Specification

Updated: 4 October 2026, strict retry handling and submission verification.

ORACZEN / TAKE-HOME B

Extraction
Workbench

Complete project specification

A full-stack workspace for turning customer support conversations into validated records that a person can inspect, correct, approve, and export.

Input	Business fields	Current checks
150 source tickets	8 structured values	55 backend + 8 frontend tests

Implementation snapshot - 4 October 2026

Python/FastAPI and Next.js App Router implement the two-service workflow. Mock remains the no-key grading default. The optional Gemini adapter has passed a live integration check using invented tickets; accuracy across the assignment dataset is not measured. Every provider output that fails validation twice now requires review, with raw attempts and usable draft fields retained.

Submission: Sunday, 4 October 2026, 12:00 PM IST. Deliver a public GitHub repository containing source code, README.md, DECISIONS.md, .env.example, and a clear history of real development commits.

Repository: [Kushagra-Kataria/oraczen-extraction-workbench](#)

Reading map

Page	Topic
2	Product scope and feature inventory
3	Ticket data and extraction schema
4-5	Pipeline, classification, uncertainty, and failure handling
6-7	Human review, CSV, architecture, and API contracts
8-10	Setup, verification, delivery, limitations, and next steps

What the product does

Primary user: a support reviewer responsible for the final structured record.

The workbench provides one workflow: load tickets, select a batch, extract proposals, review uncertain results, save corrections, approve valid records, and export them. It processes the supplied dataset; it is not a help-desk system for receiving new customer messages.

Capability	Implemented behavior
Ticket inbox	Lists all 150 tickets with subject, body preview, channel, date, and source ID.
Search and selection	Searches ID, subject, body, and sender. Filter by channel; select a subset, filtered rows, or the complete dataset. Selection survives filters.
Background extraction	Starts a job immediately, caps simultaneous provider calls, and processes tickets independently.
Live progress	Shows queued/running/processed/failed counters and per-item state. Results appear as items finish, before the whole batch ends.
Validation and retry	Validates JSON and all business fields. One repair attempt receives actual validation errors. Invalid drafts keep raw attempts.
Review workspace	Displays original text and editable proposals together, field evidence, missing/inferred markers, warnings, and field-specific errors.
Corrections and approval	Validated partial edits, human provenance, stale-version protection, and explicit whole-record approval.
Queue and export	Needs-review sorting; filters for human-edited, reviewed, and failed records. One-click CSV exports approved, valid records.
Cancel and draft guards	Cancels unfinished work while retaining results; polling preserves unsaved edits, and record switches require save/discard.

Two interchangeable providers

Mock provider: the deterministic local extraction engine required for no-key grading. Gemini provider: an optional real LLM used when configured. Both use the same job, validation, retry, review, and export pipeline. Select any subset or the complete source dataset; selecting tickets does not switch providers.

Data and business contract

The original source is data/tickets.jsonl: UTF-8 JSON Lines, not a JSON array.

Each of the 150 lines contains one ticket. Input fields are id, subject, body, channel, received_at, from_email, and attachments. Startup rejects malformed records or duplicate IDs. The app reads the source without rewriting it. Attachments are counts only; their contents are unavailable.

Field	Required?	Type and accepted values
company	Yes	Nonblank string; trimmed; maximum 200 characters.
product	Yes	Zen Orchestrator Zen Studio Zen Connect Zen Insights Zen Vault
category	Yes	outage billing bug feature_request how_to churn_risk
severity	Yes	low medium high critical
requested_action	Yes	refund credit fix callback information none
refund_amount	No	Finite, nonnegative JSON number in USD; null when unresolved or absent.
deadline	No	Valid calendar date in YYYY-MM-DD format; null when unresolved or absent.
escalated	Yes	Actual JSON boolean: true or false. Strings/numbers are rejected.

Review metadata is separate from the business schema

Each result also stores record/job/ticket IDs, status, schema validity, approval, attempt count, raw outputs, current errors, original notes, version, and per-field source/grounding/evidence. A partial draft can hold null required values for editing; the strict business model still rejects those values until they are repaired.

Observed source complexity

Channels: 45 email, 42 chat, 34 web_form, and 29 phone_transcript. Twenty-five tickets count attachments. Bodies include quoted replies, signatures, legal footers, French text, typos, sparse content, relative dates, and spoken amounts. Sender domains and body companies can conflict; channel labels do not reliably describe body format.

How extraction runs

The same scheduling, validation, retry, and review pipeline serves both providers.

Select -> schedule -> provider -> validate -> store -> human review -> CSV.

Step	System behavior
1. Validate request	Accept 1-150 ticket IDs. Reject unknown IDs and duplicates with HTTP 422.
2. Start a job	POST /api/jobs returns HTTP 202 with the job ID before extraction completes.
3. Bound concurrency	Tracked asynchronous tasks share one semaphore across jobs. Default cap: four provider calls.
4. Skip sparse input	Bodies exactly '?' or 'please advise' create incomplete review drafts without provider calls.
5. Validate output	Check the proposal envelope, each business field, and exact source-quote matches.
6. Retry or finalize	Retry once with actual validation errors. Every second invalid output becomes needs_review, including optional values, unknown fields, JSON, and envelope failures. Retain both raw outputs, final errors, and individually valid draft fields.
7. Isolate failures	Per-attempt timeout or provider exceptions fail one item; other tickets continue.

Completion and approval are different

A job is done when all items have reached a terminal state. An extracted item is done only when a provider output passes the complete schema. Inference and ambiguity notes alone do not force review. Every second invalid output needs review; a sanitized draft is not automatically accepted. Needs-review items count as processed. A person must approve every record intended for export.

```
queued + running + done + failed = total
done includes needs_review; needs_review is a reported subset
```

Extraction and classification rules

Current mock behavior is conservative text matching; it is not an LLM accuracy benchmark.

The mock removes quoted reply lines before most classification. It finds product names and company signatures; a sender-domain fallback is visibly inferred. Categories use ordered phrase rules, preferring the body over the subject: churn risk, billing, outage, feature request, bug, then how-to. This prioritization resolves one category from a schema that cannot represent several issues.

Severity uses explicit phrases such as urgent, critical, scheduled-job failure, blocker, and workaround. Otherwise it remains missing. Requested actions use refund, credit, call, fix, and information cues; agent lines in phone transcripts are excluded so agent suggestions do not override the caller. Refunds require an exact sole USD amount; only explicit ISO deadlines are extracted. Escalation uses escalation or leadership cues.

Case	Chosen behavior
Missing severity	Do not invent impact. Leave it missing and require review, even after the repair attempt.
French + EUR (0058)	Keep the original EUR amount in notes, leave USD refund empty, and flag the sender/company conflict.
Multiple issues (0089)	Choose churn_risk for the renewal threat; preserve other problems in notes. Ambiguous dates and amounts remain unresolved.
Typos (0105)	Normalize 'zen studioo' to Zen Studio with a visible warning; missing severity still needs review.
Spoken amount (0131)	Classify the caller's accepted action as credit; do not fabricate an exact amount from 'nine thousand something'.
Deliberate mock failures	0005 returns invalid severity on attempt one and repairs it. 0003 returns an invalid product on both attempts.

Why some results require review

Missing required values or any other validation failure trigger one repair. If the second output still fails, the item requires review. Missing optional values are allowed; malformed optional values are rejected. Notes about conflicts, normalization, currency, dates, or multiple issues remain visible without changing an otherwise valid output's status. action=none and escalated=false are valid values. Quote matching proves presence, not correct interpretation.

Gemini receives source instructions, a JSON schema, and repair feedback. Remote proposals may contain nulls; local Pydantic validation remains authoritative. Grounding flags and notes expose uncertainty. A twice-rejected draft retains errors and is not marked schema-valid until a human save/approval validates it. The reviewer may correct an optional value or explicitly accept leaving it empty; export still requires approval.

Human review and final export

A reviewer owns the final decision; schema validity and provenance remain visible.

Reviewer action	Result
Select a record	Raw source and proposal stay on one page. Desktop uses side-by-side panels; smaller screens stack them.
Focus a field	Its exact evidence quote is highlighted when present in the body; missing/inferred/human-edited indicators remain visible.
Inspect a failure	View per-field errors, original notes, and raw provider attempts without losing the salvageable draft.
Save changes	Submit only changed fields. Valid partial repairs can be saved while unrelated required fields remain missing.
Approve and save	Validate the entire record. Reject invalid approval with field-specific errors; valid approval makes the record export-eligible.
Edit an approved record	Mark supplied fields as human-edited. Clear previous approval unless approval is explicitly requested again.
Save an old version	HTTP 409 prevents silent overwrite of a newer record. Refresh and reconcile the draft.
Switch or cancel	Unsaved edits require save/discard before record switching. Cancellation retains completed records and fails unfinished items with notes.

CSV contract

The export contains only `reviewed=true` and `schema_valid=true` records. Headers are `ticket_id`, `company`, `product`, `category`, `severity`, `requested_action`, `refund_amount`, `deadline`, `escalated`, and `human_edited_fields`. Human-edited field names are separated by semicolons. Missing optional values are blank, dates are ISO, booleans lowercase, and standard CSV quoting preserves commas, quotes, and newlines. Formula-like text is prefixed with an apostrophe. With no approvals, the file contains headers only.

State protection and limits

Polling does not replace a dirty local draft; older responses cannot overwrite a newer save. Browser reload/close warns on dirty input. Internal navigation away from the job page can still discard a draft. Original attempts and current human field provenance are retained, but a durable history of every edit is not implemented.

Architecture and API

Two local services communicate over HTTP. One backend worker owns process-local state.

Layer / source	Responsibility
frontend/ - Next.js, React, TypeScript	App Router pages and root layout; client selection, polling, drafts, evidence display, corrections, and responsive CSS.
backend/app/routes.py	FastAPI HTTP boundaries, ID checks, status/results, corrections, cancellation, and reviewed CSV.
schemas.py + review.py	Strict Pydantic contracts, partial-field validation, versions, approval, and provenance.
jobs.py + store.py	Tracked tasks, global semaphore, counter arithmetic, cancellation, and in-memory dictionaries.
extraction.py + providers/	Shared validation/retry/grounding pipeline; interchangeable mock and Gemini adapters.
config.py + tickets.py	Backend/root .env settings, secret handling, stable paths, and JSONL loading.

Method and route	Request / response purpose
GET /api/health	Health and selected provider.
GET /api/tickets	Original ticket list.
POST /api/jobs	{ticket_ids: [...]} -> HTTP 202 job snapshot.
GET /api/jobs/{id}	Job state, queued/running/done/failed counts, and per-item state.
GET /api/jobs/{id}/results	Available records, metadata, and original tickets.
POST /api/jobs/{id}/cancel	Stop unfinished work; return updated status.
PATCH /api/records/{id}	{version, fields, reviewed?}; validated correction or 422/409.
GET /api/jobs/{id}/export.csv	CSV attachment containing reviewed, valid records.

Next.js rewrites /api requests to FastAPI in development and production. App Router defines / and /jobs/[id], including direct visits to job URLs and real unknown-page responses. The dynamic server route awaits its parameters and passes the ID to a client workbench. Polling reads progress then results about once per second, stops at terminal state, and aborts on unmount. Cloud deployment is outside scope.

Installation and provider setup

Requirements: Git, Python 3.12+, Node.js 22.12+, npm, and network access for package installation.

Clone the public repository and open two terminals at its root. Defaults work without a .env file. The commands below are for Windows PowerShell; macOS/Linux uses python3, .venv/bin/python, and npm.

Terminal 1 - backend

```
cd backend
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Terminal 2 - frontend

```
cd frontend
npm.cmd ci
npm.cmd run dev
```

Open <http://127.0.0.1:5173>. API documentation: <http://127.0.0.1:8000/docs>. Keep both processes running; use Ctrl+C to stop. Run exactly one backend worker.

Configuration variable	Default / purpose
EXTRACTION_PROVIDER	mock; select gemini for real hosted LLM calls.
GEMINI_API_KEY	Empty; a personal server-only key is required in Gemini mode.
GEMINI_MODEL	gemini-3.1-flash-lite; configurable model identifier.
MAX_CONCURRENCY	4; cap 1-20 simultaneous provider calls across jobs.
MOCK_DELAY_MS	650; artificial delay per attempt, 0-10000 ms.
PROVIDER_TIMEOUT_SECONDS	30; per-attempt timeout, greater than 0 and at most 120.
TICKETS_PATH	data/tickets.jsonl; root-relative or absolute dataset path.
BACKEND_URL	http://127.0.0.1:8000 ; Next.js API rewrite target in development and production. Set in the frontend process, frontend/.env.local, or root .env before building/starting.

For Gemini: copy .env.example to backend/.env, set EXTRACTION_PROVIDER=gemini and your key, then restart FastAPI. Selected tickets are sent to Google's API. A concurrency cap does not enforce requests-per-minute limits. Secrets stay server-only; never use NEXT_PUBLIC_ for a key. To serve a production frontend, run npm.cmd run build then npm.cmd run start, with FastAPI still running. API rewrite targets are fixed by the build.

Verification and acceptance criteria

Checks demonstrate workflow behavior; they do not establish real-model extraction accuracy.

Evidence	Observed result / scope
Backend suite	55 passing tests: validation, retry feedback, completion, progress, concurrency, cancellation, timeout, corrections, versions, CSV, simulated Gemini, strict optional/extra/envelope failures, and human approval of a retained draft.
Frontend suite	8 passing tests after full-source selection was added: filtering, batch submission, incremental results, field errors, provenance, payload types, and draft protection.
Static checks	Ruff lint/format checks passed; frontend Prettier, TypeScript checking, and production build passed for their latest relevant changes.
Offline full batch	150/150 terminal results, 0 provider failures: 22 done and 128 needs_review under current mock rules. Those numbers describe routing, not accuracy or approvals.
Live Gemini check	Invented outage/billing examples and sparse input verified extraction, USD refund parsing, correction, approval, version checks, and CSV. No assignment tickets were sent.
Browser workflow	Ticket selection, early results, correction, invalid approval feedback, provenance, approval, actual CSV download, and responsive layout checked.
Clean-clone installation	Source commit 2f91ddb installed without copied .env or packages. 55 backend/eight frontend tests, lint, formatting, TypeScript, and Next.js build passed. No-key HTTP checks covered early results, progress, repair/review, corrections, provenance, versions, CSV, and dynamic/unknown routes.

Assignment acceptance checklist

The grader can start without a key, select tickets, receive HTTP 202, watch bounded processing, and inspect early results. Invalid output triggers one repair; every second validation failure leaves a reviewable draft and raw outputs. Edits are validated and distinguishable; only approved valid records export. Setup and decision notes are present. Next.js App Router meets the frontend stack requirement.

How to reproduce checks

```
# From backend/
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\ruff.exe check app tests
# From frontend/
npm.cmd test
npm.cmd run format:check
npm.cmd run build
```

Delivery, limitations, and next work

The assignment deliverable is the repository; this PDF is a companion specification.

Deliverable	Where / purpose
Source code	backend/ and frontend/; original data/tickets.jsonl preserved.
README.md	Prerequisites, installation, architecture, usage, API, tests, configuration, and troubleshooting.
DECISIONS.md	Five required product decisions, data observations, provider choice, trade-offs, and another-day improvements.
.env.example	All configurable values, empty key placeholders, no sensitive credentials.
Git history	Small real development commits, including the frontend migration; no fabricated dates or reconstructed history.
Interview preparation	docs/INTERVIEW_GUIDE.md explains reading order, ticket lifecycle, async tasks, versions, and failure behavior.

Required stack and retry behavior

The frontend uses Next.js App Router, React, and TypeScript. Vitest uses Vite only as a test engine. Every provider output failing validation twice becomes `needs_review`, including optional fields and unsupported properties. Raw attempts and valid draft fields remain available for correction. The deterministic mock and deliberate failure paths remain part of no-key grading.

Current limits

The mock uses narrow text rules and leaves many missing facts unresolved. Gemini integration is verified on invented examples; assignment accuracy is unevaluated. In-memory jobs, edits, approvals, and versions disappear on backend restart; multiple workers would disagree. There is no database, authentication, durable audit log, provider RPM limiter, or semantic accuracy evaluation. Attachment contents are unavailable. Internal navigation can discard drafts.

Implementation priorities after this snapshot

Evaluate labeled extraction accuracy with approved data, then add warning codes, rate-limit backoff, persistence, and edit-event history. Keyboard-first review, single-record reruns, richer currency/date handling, Docker Compose, and streaming progress remain deferred. Multi-user production deployment needs additional design.

Submission and interview preparation

Recheck a fresh clone of the final revision, confirm public repository access, and send the URL before 4 October 2026 at 12:00 PM IST. Read the implementation before submitting: explain one ticket from input to CSV, why missing facts require review, and what breaks when validation, the semaphore, or version checks are removed.

Source of this specification: ASSIGNMENT.md, DATA.md, README.md, DECISIONS.md, backend/app modules, frontend/src modules, automated test results, and recorded local/browser verification. Provider API reference: ai.google.dev/api/generate-content.